CLASE 7 · 3 HS

Estado y Secuenciales

Que el programa **recuerde** lo que pasó antes: flip-flops, contadores, registros y pulsadores que responden un apretón = una acción, sin **delay()**.

Programación en C con ESP32 · Sistemas Lógicos · Universidad Nacional de Santiago del Estero · 2026

HOY

Ocho bloques, un programa que recuerda

1 · 15 min

La memoria

2 · 25 min

Local, global y static

3 · 25 min

Detectar el flanco

4 · 25 min

Rebote y millis()

5 · 10 min

seApreto() para tres botones

6 · 20 min

millis() contra delay()

7 · 25 min

Galería de secuenciales

8 · 35 min

Ejercicios y TP

Secuencial = necesita memoria

COMBINACIONAL

El timbre

Suena **mientras** se lo aprieta. La salida depende solo de las entradas **actuales**: alcanza con una tabla.

SECUENCIAL

El velador con pulsador

Un apretón lo enciende, otro lo apaga: tiene que **recordar** si estaba encendido.

El estado de un circuito es lo que guardan sus flip-flops; el de un programa, lo que guardan sus variables. Un flip-flop = un **bool**. Un registro de 8 = un **uint8_t**.

La local se olvida, la global recuerda

LOCAL · ADENTRO DE loop()

```
void loop() {
  int contador = 0;
  contador++;
  Serial.printf("%d\n", contador);
}
```

1, 1, 1, 1… ¡siempre 1!

GLOBAL · AFUERA DE TODO

```
int contador = 0;
void loop() {
  contador++;
  Serial.printf("%d\n", contador);
}
```

1, 2, 3, 4… ¡recuerda!

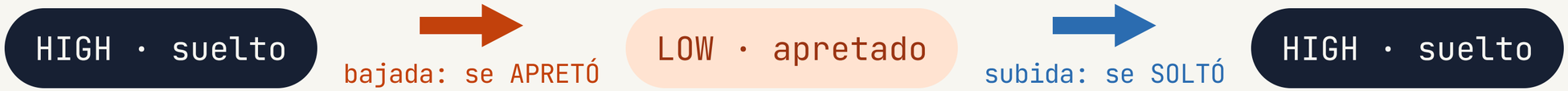
loop() es una función que se llama una y otra vez: todo lo local a loop() se crea al entrar y **se destruye al salir**.

Alcance y vida: tres formas de declarar

Tipo	Dónde se declara	Quién la usa	Cuánto vive	¿Recuerda?
Local	Adentro de una función	Solo esa función	Mientras se ejecuta la función	**No**
Global	Afuera de todo	Todas las funciones	Todo el programa	**Sí**
Local static	Adentro, con static	Solo esa función	Todo el programa	**Sí**

static = alcance de local + vida de global. En el curso, el estado va en **globales**, arriba, juntas y con nombres claros: se ve de un vistazo qué recuerda el programa.

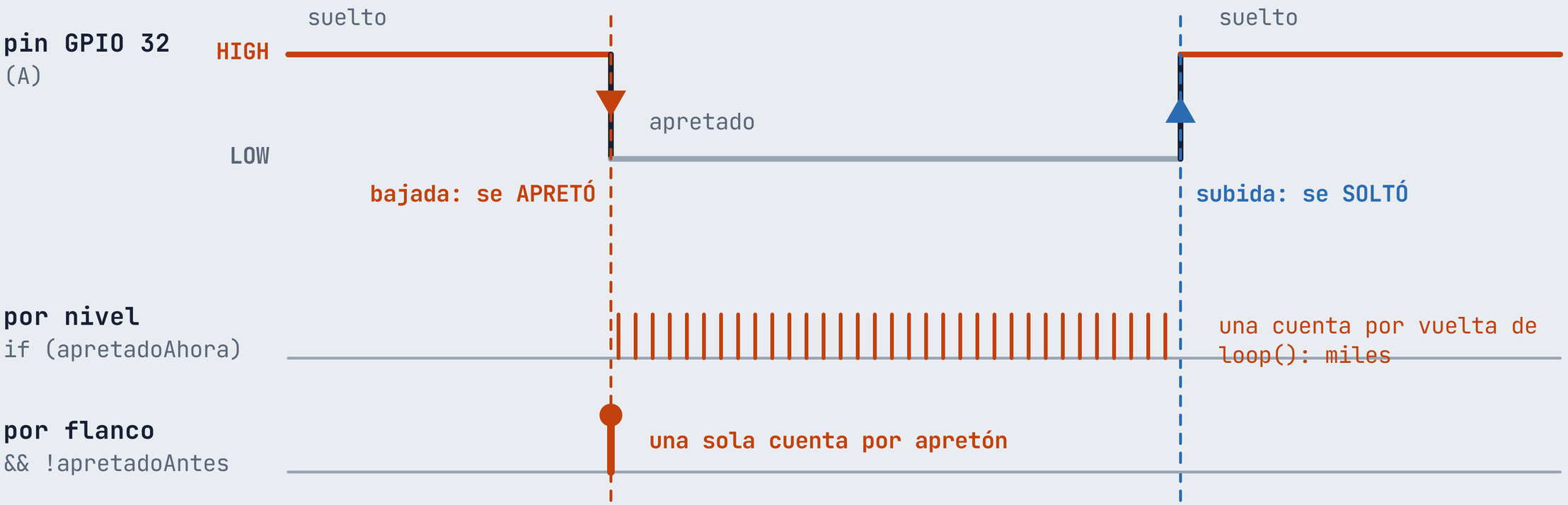
Nivel contra flanco



apretadoAntes	apretadoAhora	Qué pasó
`false`	`false`	Sigue suelto
`false`	`true`	**Se apretó** (flanco de bajada del pin)
`true`	`true`	Sigue apretado
`true`	`false`	**Se soltó** (flanco de subida del pin)

Para saber si algo **cambió** hay que comparar cómo está ahora con cómo estaba antes. Y eso necesita memoria: una variable global.

La señal del botón en el tiempo



Con **INPUT_PULLUP**, apretar es un **flanco de bajada**: el instante en que el pin pasa de HIGH a LOW.

Recordar cómo estaba antes

```
void loop() {
  bool apretadoAhora = (digitalRead(BOTON_A) == LOW);
  if (apretadoAhora) {
    porNivel++;          // miles por segundo
  }
  if (apretadoAhora && !apretadoAntes) {
    porFlanco++;         // uno por apretón
  }
  apretadoAntes = apretadoAhora;
}
```

Probarlo

Mantener A apretado: **porNivel** sube a miles, **porFlanco** sube 1.

La última línea anota para la próxima vuelta: **apretadoAntes** es la memoria.

Un apretón, varios flancos

El rebote mecánico

Las chapitas rebotan de **1 a 10 ms** antes de quedar firmes. El ESP32 ve varios flancos: el contador salta de a 2 o de a 3.

`millis()`

Cuántos milisegundos pasaron desde que se encendió el ESP32. **No espera nada**: consulta la hora y sigue.

Detalle	Por qué
Se guarda en **unsigned long**	Entero sin signo de 32 bits: llega a ≈ 49,7 días. En un int se desborda a los ~24 días y da negativos.
Se imprime con **%lu**	l de long, u de unsigned.
Para medir tiempo, **se resta**	`millis() - ultimoCambio`

El rebote, visto de cerca



Antirrebote de 50 ms

```
void loop() {
  bool apretadoAhora = (digitalRead(BOTON_A) == LOW);
  if (apretadoAhora != apretadoAntes) {  // ¿cambió?
    if (millis() - ultimoCambio >= ANTIRREBOTE_MS) {
      ultimoCambio = millis();
      apretadoAntes = apretadoAhora;
      if (apretadoAhora) {
        contador++;  // apretón de verdad
      }
    }
  }
}
```

La idea

Se acepta un cambio solo si pasaron **50 ms** desde el último cambio aceptado.

Los rebotes llegan todos juntos: quedan afuera.

Globales: **apretadoAntes**, **ultimoCambio** y **ANTIRREBOTE_MS**.

Siempre restar

BIEN

`millis() - ultimaVez ⩾ intervalo`

Sigue funcionando cuando millis() da la vuelta.

MAL

`millis() ⩾ ultimaVez + intervalo`

Falla justo al desbordar, a los 49,7 días.

Si millis() fuera un **uint8_t** (0 a 255): ultimoCambio = 250, intervalo = 50.

millis()	Con resta	Con suma (250 + 50 = 300 → 44)
251	251 − 250 = 1 → 1 ≥ 50: falso. Bien	251 ≥ 44: verdadero. **¡Falla!**
14	14 − 250 = 20 → 20 ≥ 50: falso. Bien	14 ≥ 44: falso
44	44 − 250 = 50 ≥ 50: verdadero	44 ≥ 44: verdadero

seApreto(): un apretón, una acción

```
bool seApreto(int boton) {
  bool apretadoAhora = (digitalRead(BOTONES[boton]) == LOW);
  bool hayFlanco = false;
  if (apretadoAhora != apretadoAntes[boton]) {
    if (millis() - ultimoCambio[boton] >= ANTIRREBOTE_MS) {
      ultimoCambio[boton] = millis();
      apretadoAntes[boton] = apretadoAhora;
      if (apretadoAhora) {
        hayFlanco = true;
      }
    }
  }
  return hayFlanco;
}
```

Reglas

Una cajita por botón: **0 = A, 1 = B, 2 = C**.

Llamarla **una sola vez por vuelta** por botón.

En setup(): **INPUT_PULLUP** para los tres.

Nunca más delay() para leer un botón.

No esperar: mirar el reloj

delay() BLOQUEA: EL BOTÓN SE MIRA CADA 2 S

```
void loop() {
  digitalWrite(LEDS[0], HIGH);
  delay(1000);
  digitalWrite(LEDS[0], LOW);
  delay(1000);
  if (seApreto(0)) { ... }
}
```

EL PATRÓN CON millis(), PARA MEMORIZAR

```
if (millis() - ultimaVez ⩾ INTERVALO) {
  ultimaVez = millis();
  // hacer la acción
}
```

Una **ultimaVez** por tarea: cada una a su ritmo, sin bloquear a las demás.

Flip-flops en C: latch SR, D y T

```
void loop() {
  if (seApreto(1)) {   // reloj B
    qD = (digitalRead(BOTON_A) == LOW);
  }
  if (seApreto(2)) {   // reloj C
    qT = !qT;
  }
  digitalWrite(LEDS[0], qD);
  digitalWrite(LEDS[7], qT);
}
```

Latch SR

A = Set, B = Reset. Responde al **nivel**: no usa seApreto().

Flip-flop D

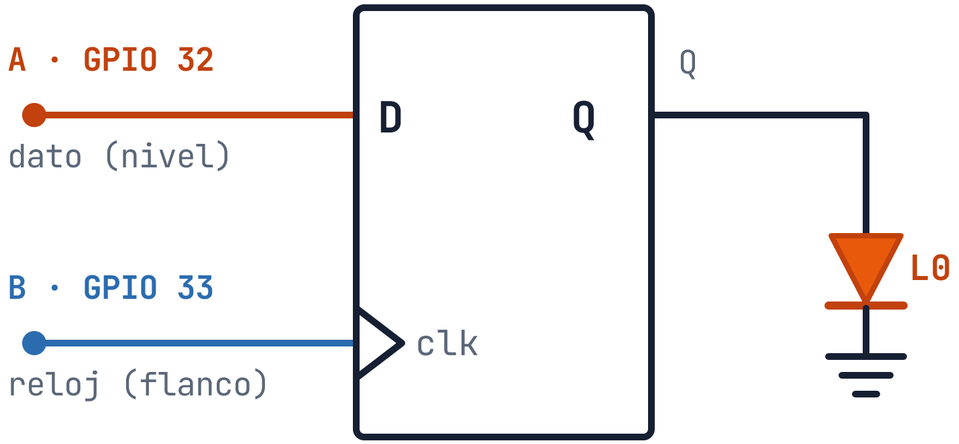
En el **flanco** de B, L0 copia lo que vale A.

Flip-flop T

Cada flanco de C **invierte** L7: el velador.

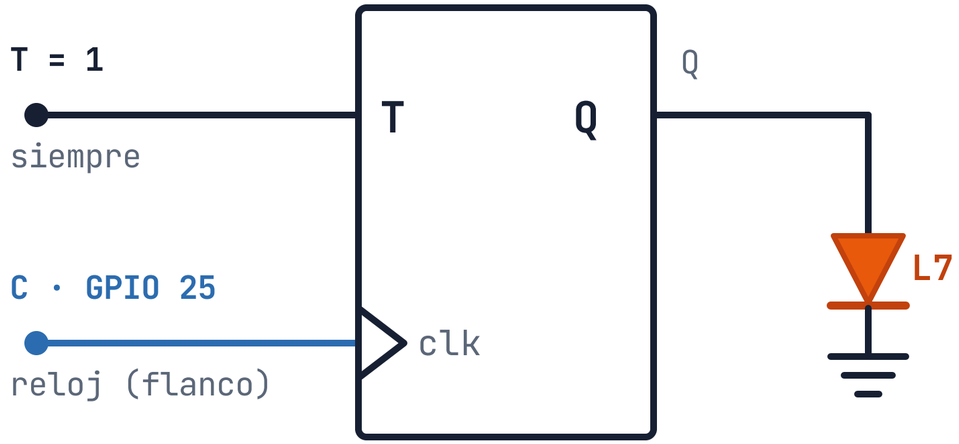
Los símbolos: flip-flops D y T

Flip-flop D



en el flanco de B:
Q copia A

Flip-flop T



en cada flanco de C:
Q se invierte

El **triángulo** en la entrada de reloj quiere decir: cambia en el **flanco**, no mientras el reloj está en 1.

Contador y registro en un uint8_t

CONTADOR · A = +1, B = -1

```
uint8_t contador = 0;
void loop() {
  if (seApreto(0)) {
    contador++;  // 255+1 = 0
    mostrarByte(contador);
  }
  if (seApreto(1)) {
    contador--;  // 0-1 = 255
    mostrarByte(contador);
  }
}
```

REGISTRO DE DESPLAZAMIENTO · A = dato, B = reloj

```
uint8_t registro = 0;
void loop() {
  if (seApreto(1)) {   // flanco del reloj
    bool dato = (digitalRead(BOTON_A) == LOW);
    registro = (registro << 1) | dato;
    mostrarByte(registro);
  }
}

// todos se corren hacia L7
// y el dato entra en L0
```




Divisor de frecuencia: contador ripple

```
void loop() {
  if (millis() - ultimoTic ⩾ MEDIO_PERIODO) {
    ultimoTic = millis();
    int i = 0;
    bool propagar = true;
    while (propagar && i < 7) {
      etapa[i] = !etapa[i];
      digitalWrite(LEDS[i], etapa[i]);
      propagar = (etapa[i] == false);
      i++;
    }
  }
  digitalWrite(LEDS[7], digitalRead(BOTON_A) == LOW);
}
```

Cada etapa divide por 2

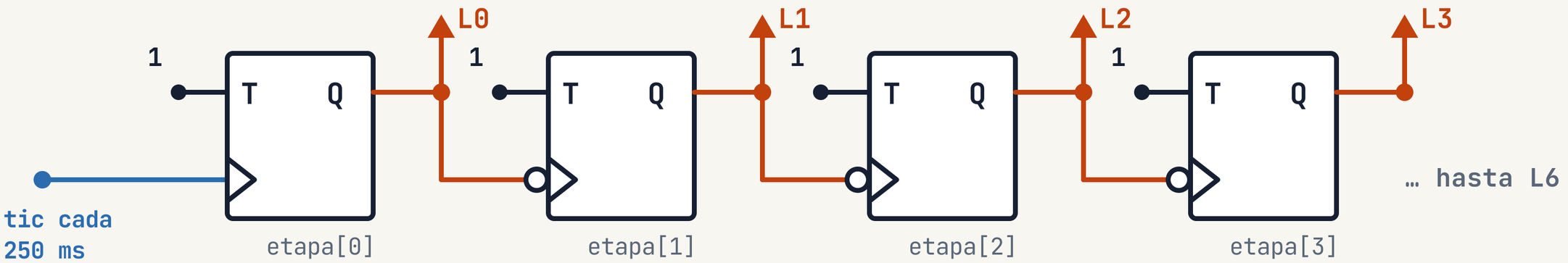
L0 a 2 Hz, L1 a 1 Hz, L2 a 0,5 Hz… Juntos, **cuentan en binario**.

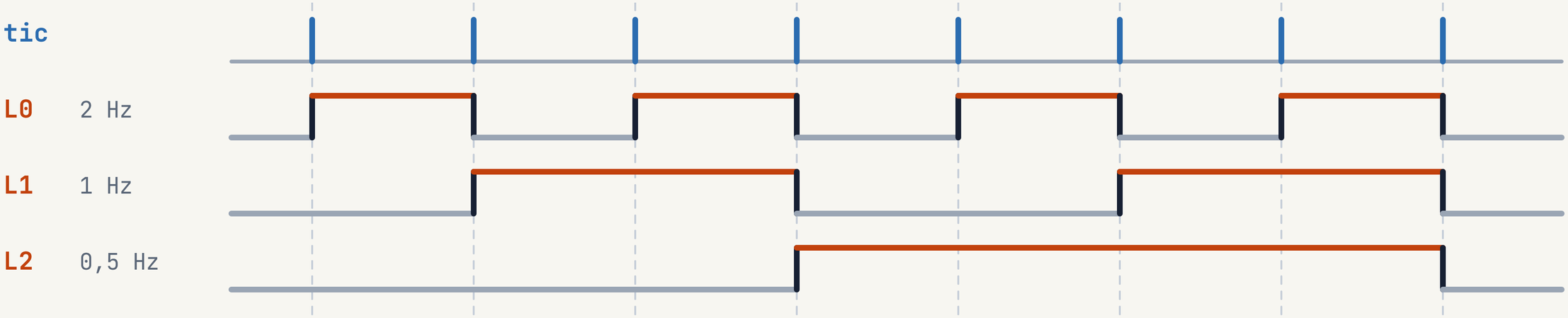
Una etapa cambia cuando la anterior **baja de 1 a 0**.

L7 sigue a A **sin demora**: nadie bloquea con delay().



Contador ripple: una cadena de flip-flops T





8 · EJERCICIOS · 35 MIN

Ahora ustedes

EJERCICIO 1

Flip-flops JK y T

C = reloj, A = J, B = K. Q en L0, Q negada en L1 y un T en L7. ¿Por qué un T es un JK con J = K?

EJERCICIO 2

Registro de desplazamiento

A = dato, B = reloj, C = borrar. Imprimir el byte en binario y escribir 10110001 bit por bit.

EJERCICIO 3

Contador BCD

A = +1, B = −1, C = reset. De 0 a 9 en L0..L3. Al pasar de 9 a 0: imprimir Acarreo! e invertir L7.

EJERCICIO 4

¿Cuánto estuvo apretado?

Al soltar A, imprimir los ms. Desafío: una barra de LEDs que crece cada 250 ms, sin delay().

TP CLASE 7

El entrenador cuenta

1. Comentario de encabezado: nombre, clase y qué hace el programa.
2. Un contador binario de 8 bits (uint8_t) que se vea en L0..L7.
3. A suma 1 y B resta 1, con flanco y antirrebote (seApreto). **Prohibido usar delay().**
4. C alterna entre MANUAL y AUTO. En AUTO suma 1 solo cada 500 ms con millis(); A y B siguen respondiendo al instante.
5. Cada vez que el contador cambia, imprimir decimal, hexa, binario y el modo.
6. Cada vez que cambia el modo, avisarlo: Modo AUTO / Modo MANUAL.

```
Contador:  42 (0x2A) 00101010 [MANUAL]
Modo AUTO
```

Entrega: carpeta **SL2026-Apellido-Clase7** en ZIP, sin la carpeta .pio.



CHECKLIST DE SALIDA

Al terminar la clase, sé…

- explicar combinacional contra secuencial, y qué tiene que ver la memoria
- decir qué pasa con una variable local, global y static entre vueltas
- detectar el flanco de un pulsador guardando su estado anterior
- explicar el rebote y cómo lo filtra el antirrebote de 50 ms
- usar millis() - ultimaVez >= INTERVALO y explicar por qué se resta
- explicar por qué delay() impide atender los botones
- programar un flip-flop T y uno D, un contador y un registro

La semana que viene: hoy hubo dos modos con un bool. ¿Y si fueran cinco? En la Clase 8, **máquinas de estado con enum y switch**, y el entrenador completo.